

TD8 - Arbre binaire de recherche (ABR)

Exercice 1 - Rappels sur les ABR

Étudier les algorithmes *successeur*, *predecesseur*, *recherche*, *insertion* et *suppression*.

Solution de l'exercice 1

Rappels sur les ABR

Soit x un noeud de l'ABR. On appelle sous-arbre gauche (respectivement sous-arbre droit) de x le sous-arbre dont la racine est le fils gauche (respectivement droit) de x .

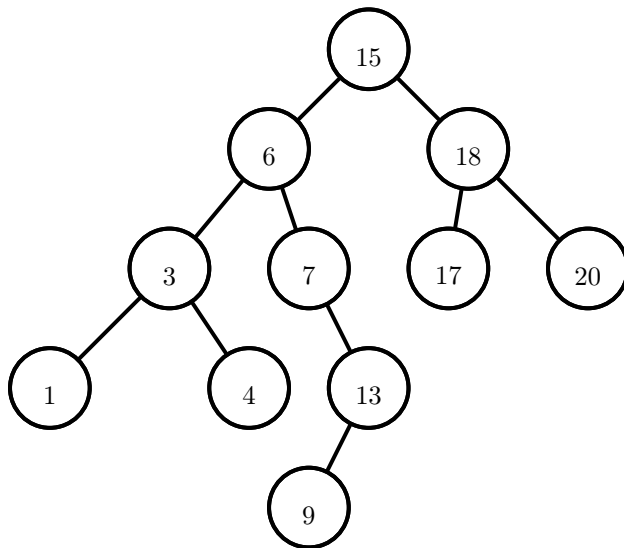
- Si $y \in A_g[x]$ alors $cle[y] < cle[x]$;
- Si $y \in A_d[x]$ alors $cle[y] > cle[x]$;

pere[x] Noeud père

gauche[x] Fils gauche

droit[x] Fils droit

cle[x] Clé



Présentation des algorithmes

Recherche, insertion, min, max Rechercher 13, 19 ; insérer 8 ; rechercher le min ; rechercher le max.

Suppression d'un noeud x

- Si x est une feuille : le noeud est directement supprimé
- Si x admet un unique fils : le fils remplace x
- Sinon
 - Soit remplacer le noeud x par le noeud min de $A_d[x]$ (son successeur)
 - Soit remplacer le noeud x par le noeud max de $A_g[x]$ (son prédécesseur)
 - Le prédécesseur/successeur supprimé admettait 0 ou 1 fils : On sait traiter ce cas

Exemple : Suppression des noeuds de clé 9, 7, 6.

Recherche du successeur (prédécesseur) d'un noeud x Le successeur (prédécesseur) est le noeud de plus petite (grande) clé telle que la clé soit supérieure (inférieure) à celle de x .

- Si x possède un sous-arbre droit (gauche) non vide, le successeur (prédécesseur) est le plus petit (grand) élément de cet arbre.
- Sinon, le successeur (prédécesseur) est le premier ascendant de x qui a x dans son sous-arbre gauche (droit).

Exemples : Successeur de 15, 13, prédécesseur 6, 9

Exercice 2 - Final Printemps 2003

Soit A un ABR et a une valeur. On veut déterminer la plus petite valeur b contenue dans A qui est supérieure ou égale à a .

Par exemple, pour A contenant les valeurs 1, 3, 5, 7, 9 : Si $a = 4$ alors $b = 5$. Si $a = 9$ alors $a = b = 9$. Si $a = 10$ alors il n'existe pas de valeur b . Un noeud de l'arbre A correspond à une structure de type *struct noeud* comportant trois champs :

- une valeur entière
- un pointeur vers le sous-arbre gauche
- un pointeur vers le sous-arbre droit

Le type *arbre* correspond à un pointeur sur la structure *struct noeud*.

1. Définir la structure *struct noeud* et le type *arbre* en C.
2. Écrire une fonction *int max(arbre A)* en C qui retourne la clé de valeur maximale contenue dans un ABR non vide, ainsi qu'une fonction *int min(arbre A)* qui retourne la clé de valeur minimale.
3. Écrire un algorithme récursif *search_geq(A : ABR, a : entier)* qui prend en entrée un ABR A et une valeur a et qui retourne le noeud de l'arbre A contenant la plus petite valeur supérieure ou égale à a (ou NIL si un tel noeud n'existe pas).
4. Écrire une fonction *search_geq(arbre A, int a)* en C qui correspond à l'algorithme de la question précédente.

Solution de l'exercice 2

```
struct noeud {
    int val;
    struct noeud * sag; // Fils gauche
    struct noeud * sad; // Fils droit
};
typedef struct noeud * arbre;
```

Définition de la structure

On renvoie un pointeur sur le noeud de l'arbre. On suppose que l'arbre n'est pas vide.

```
int max_it(arbre a) {
    while(a->sad!=NULL) {
        a=a->sad;
    }
    return a->val;
}
int max_rec(arbre a) {
    if(a->sad!=NULL) {
        return max_rec(a->sad);
    } else {
        return a->val;
    }
}
int min_it(arbre a) {
    while(a->sag!=NULL) {
        a=a->sag;
    }
    return a->val;
}
```

```

int min_rec(arbre a) {
    if(a->sag!=NULL) {
        return min_rec(a->sag);
    } else {
        return a->val;
    }
}

```

Algorithmes max et min

Dans le cas où $cle[x] > a$, le résultat est soit x , soit un noeud du sous-arbre gauche de x :

- Si $search_geq(Ag[x], a)$ renvoie $s=NIL$, c'est x
- Si $search_geq(Ag[x], a)$ renvoie $s \neq NIL$, $a \leq cle[s] < cle[x]$, c'est s

```

search_geq(A:ABR; a:entier)
x:Noeud:=racine[A]
Si(x=NIL)
    retourner NIL
Sinon si(cle[x]=a)
    retourner x
Sinon si(cle[x]<a)
    retourner search\_geq(Ad[x], a)
Sinon
    s:Noeud:=search\_geq(Ag[x], a)
    Si(s=NIL)
        retourner x
    Sinon
        retourner s

```

Algorithme de recherche du successeur

```

struct noeud * search_geq(arbre A, int a) {
    // Renvoie NULL si a n'a pas de successeur
    // Renvoie le noeud courant si sa valeur est a
    if(A==NULL || A->val==a) {
        return A;
    } else if(A->val < a) {
        // Le successeur est dans le sous-arbre droit
        return succ(A->sad, a);
    } else {
        // Le successeur est soit le noeud courant, soit dans le sous-arbre
        gauche
        // Peut-etre que le sous-arbre gauche est vide ou ne contient que des
        valeurs plus petites que a
        struct noeud * s = succ(A->sag, a);
        return (s == NULL ? A : s);
    }
}

```

Implémentation de la recherche

Exercice 3 - Intervalles et modélisation avec un ABR

On s'intéresse à la gestion informatique des réservations sur l'année d'une salle de conférence. La salle ne peut être réservée que par journées. Une réservation peut être assimilée à un intervalle de jours $I = [min(I), max(I)]$, où $min(I)$ est le jour d'arrivée des participants et $max(I)$ le dernier jour de la conférence. Pour un intervalle I , on a trois opérations de base : $ajouter(I)$, $supprimer(I)$ et $intersecte(I)$ (qui renvoie VRAI lorsque la salle ne peut pas être réservée pendant l'intervalle I).

1. Définir les propriétés d'un ABR pour représenter les différentes réservations.
2. Ajouter graphiquement les réservations successives [37, 39], [3, 7], [100, 120], [50, 68], [10, 30], [1, 2], [8, 9].

3. Supprimer graphiquement la réservation [3, 7].
4. Écrire l'algorithme *intersecte* et le prouver. Analyser sa complexité.
5. Déterminer la complexité des trois opérations de base lorsque les réservations sont gérées avec un ABR, une liste chaînée triée et une liste chaînée quelconque. Comparer.

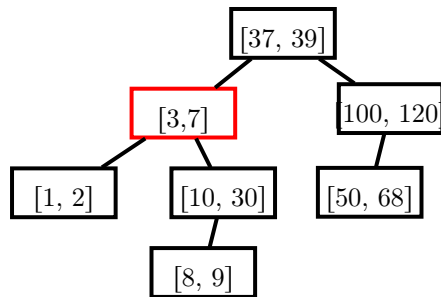
Solution de l'exercice 3

Définition de l'ABR

On définit un ABR pour représenter la gestion des réservations :

- Chaque noeud x a pour clé un intervalle I_x de réservation défini par deux valeurs $\min(I_x)$ et $\max(I_x)$
- $I_a < I_b \iff \max(I_a) < \min(I_b)$

Ajout et suppression de réservations



Lorsqu'on supprime l'intervalle [3, 7], on a deux possibilités.

- Remplacer par le min du sous-arbre droit : [8, 9]
- Remplacer par le max du sous-arbre gauche : [1, 2]

Algorithme intersecte

```

intersecte(I: intervalle; A: ABR)
  x:noeud:=racine[A]
  Si x=NIL
    retourner faux
  Sinon si max(Ix) < min(I) // [Ix][I]
    retourner intersecte(I, Ad[x])
  Sinon si max(I) < min(Ix) // [I][Ix]
    retourner intersecte(I, Ag[x])
  Sinon
    retourner vrai
  
```

Algorithme intersecte

Preuve de l'algorithme et complexité

La preuve d'un algorithme se fait en 2 étapes :

Terminaison L'algorithme se termine (il retourne un résultat)

Validité Il résout le problème (le résultat est bon)

Terminaison et complexité

Lors d'un appel à la fonction intersecte pour un arbre de hauteur k :

- Soit l'algorithme se termine
- Soit un appel récursif est réalisé avec un arbre de hauteur $k - 1$

Pour un arbre initialement de hauteur h , dans le pire des cas, après h appels, l'arbre passé en paramètre est vide et l'algorithme se termine. Complexité $O(h)$.

Validité

Montrons par récurrence sur la hauteur de l'arbre la propriété suivante : "Pour un arbre A de hauteur h , $\text{intersecte}(I, A)$ renvoie vrai si I intersecte un noeud de A et faux sinon "

Initialisation Pour un arbre A de hauteur $h = 0$

- Si A intersecte I , l'intervalle I_x de la racine intersecte I et l'algorithme renvoie vrai.
- Sinon, l'intervalle I_x n'intersecte pas I et un appel récursif est réalisé en passant un sous-arbre vide. L'algorithme renvoie alors faux.

Hérédité Pour un arbre A de hauteur $k + 1, k \geq 0$

Si A intersecte I :

- Soit le noeud x racine de A intersecte I et l'algorithme renvoie vrai directement
- Soit la clé I est plus petite que I_x . I intersecte donc un noeud de $A_g[x]$. Or l'algorithme appelle $\text{intersecte}(I, A_g[x])$ avec $A_g[x]$ arbre de hauteur k intersectant I .
- Soit la clé I est plus grande que I_x . I intersecte donc un noeud de $A_d[x]$. Or l'algorithme appelle $\text{intersecte}(I, A_d[x])$ avec $A_d[x]$ arbre de hauteur k intersectant I .

Dans les 2 cas, l'algorithme appelle intersecte avec en paramètre un arbre de hauteur k intersectant I . Par hypothèse de récurrence, l'algorithme renvoie vrai.

Si A n'intersecte pas I :

- Soit la clé I est plus petite que I_x . L'algorithme appelle $\text{intersecte}(I, A_g[x])$ avec $A_g[x]$ arbre de hauteur k n'intersectant pas I .
- Soit la clé I est plus grande que I_x . L'algorithme appelle $\text{intersecte}(I, A_d[x])$ avec $A_d[x]$ arbre de hauteur k n'intersectant pas I .

Dans les 2 cas, l'algorithme appelle intersecte avec en paramètre un arbre de hauteur k n'intersectant pas I . Par hypothèse de récurrence, l'algorithme renvoie faux.

Comparer les complexités

	Liste quelconque	Liste triée	ABR
Ajouter	$O(1)$	$O(n)$	$O(h)$
Supprimer	$O(n)$	$O(n)$	$O(h)$
Rechercher	$O(n)$	$O(n)$	$O(h)$
Intersecte	$O(n)$	$O(n)$	$O(h)$

Dans le cas général, on peut avoir un ABR de hauteur n (en insérant les éléments par ordre croissant par exemple). Dans le cas d'un arbre complet, $h = \log_2(n)$.